



ECOLE MAROCAINE DES  
SCIENCES DE L'INGENIEUR  
*Membre de* 

---

HONORIS UNITED UNIVERSITIES

# Les Bases de Données NoSQL « MongoDB »

Pr. Ilyas BENTACHFINE

[i.bentachfine@emsi.ma](mailto:i.bentachfine@emsi.ma)

- I. Généralité sur la BD MongoDB
- II. Mise en Place & Installation
- III. Interrogation de la BD :
  - 1. Opérations de base sur le shell
  - 2. Opérations avancées sur le shell



# Généralité sur MongoDB



mongoDB

## MongoDB : Généralité

- Système de gestion de base de données NoSQL orientée documents créée en 2007 (SGBD NoSQL le plus populaire).
- Disponibilité de plusieurs fonctionnalités SQL dans le langage de requête de MongoDB (count, group by, order by, sum...)
- Enregistre toutes nos données, saisies sous format JSON, dans le format binaire BSON et retourne des résultats sous format JSON.
  - ⇒ MongoDB offre aux utilisateurs la facilité d'utilisation et la flexibilité des documents JSON ainsi que la vitesse et la richesse d'un format binaire.



## MongoDB : Généralité

**Voici comment se présente une collection.**

- Chaque **document** possède un **id** attribué par l'utilisateur ou par MongoDB (ObjectId).
- Sur chaque document, les éléments peuvent être ajoutés et modifiés selon nos besoins, sauf le champ **\_id**.
- Une **collection** peut avoir d'autres documents dont les **structures se diffèrent** les unes des autres.

```
{
  "_id" :ObjectId("4efa8d2b7d285bac101e4ac7"),
  "nom" : "Pierrot",
  "prenom" : "lelion",
  "tel" : "995555",
  "enfants":
  [
    "jean-jaques",
    "alfred",
    "alain",
    "gustave",
    "jesica"
  ]
}
{
  "_id" :ObjectId("4efa8d8a8d285bac101e4ba7"),
  "nom" : "Didier",
  "prenom" : "Lefevre",
  "tel" : "66666",
  "adresse": "Rue du chemin"
}
```

- Une **collection** est l'équivalent d'une **table** en SQL.
- Un **document** est l'équivalent d'un **tuple** en SQL.
- Les types de données sous MongoDB:
  - Number
  - Boolean
  - String
  - Array
  - Object
  - Date



# Mise en Place d'une BD MongoDB



mongoDB

## Installation & Connexion à MongoDB

### ➤ Etape 1: Installation de Java

MongoDB nécessite l'installation du Java => Il faut installer le JDK. La dernière version est JDK 1.8 (Téléchargeable via le lien ci-dessous).

<http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html>

**NB:** Choisissez votre configuration et Accepter la licence pour télécharger le fichier d'installation.





## Installation & Connexion à MongoDB

### ➤ Etape 2: Téléchargement & Installation de MongoDB

- Télécharger le msi de MongoDB à partir du lien ci-dessous:  
<https://www.mongodb.com/download-center/community>
- Après avoir terminer l'installation, ajouter le chemin du bin de MongoDB au variable d'environnement *Path* pour le lancer directement dans l'invite de commande:

**Exemple:**

C:\Program Files\MongoDB\Server\4.2\bin



## Installation & Connexion à MongoDB

- **Etape 3:** (Juste pour les anciennes versions de MongoDB)
  - Ajouter le répertoire **C:/data/db** qui va recevoir les bases de données créer par l'utilisateur.
  - Pour les nouvelles versions, ce répertoire se crée automatiquement lors de l'installation de la BD:  
(C:\Program Files\MongoDB\Server\4.2\data)



## Installation & Connexion à MongoDB

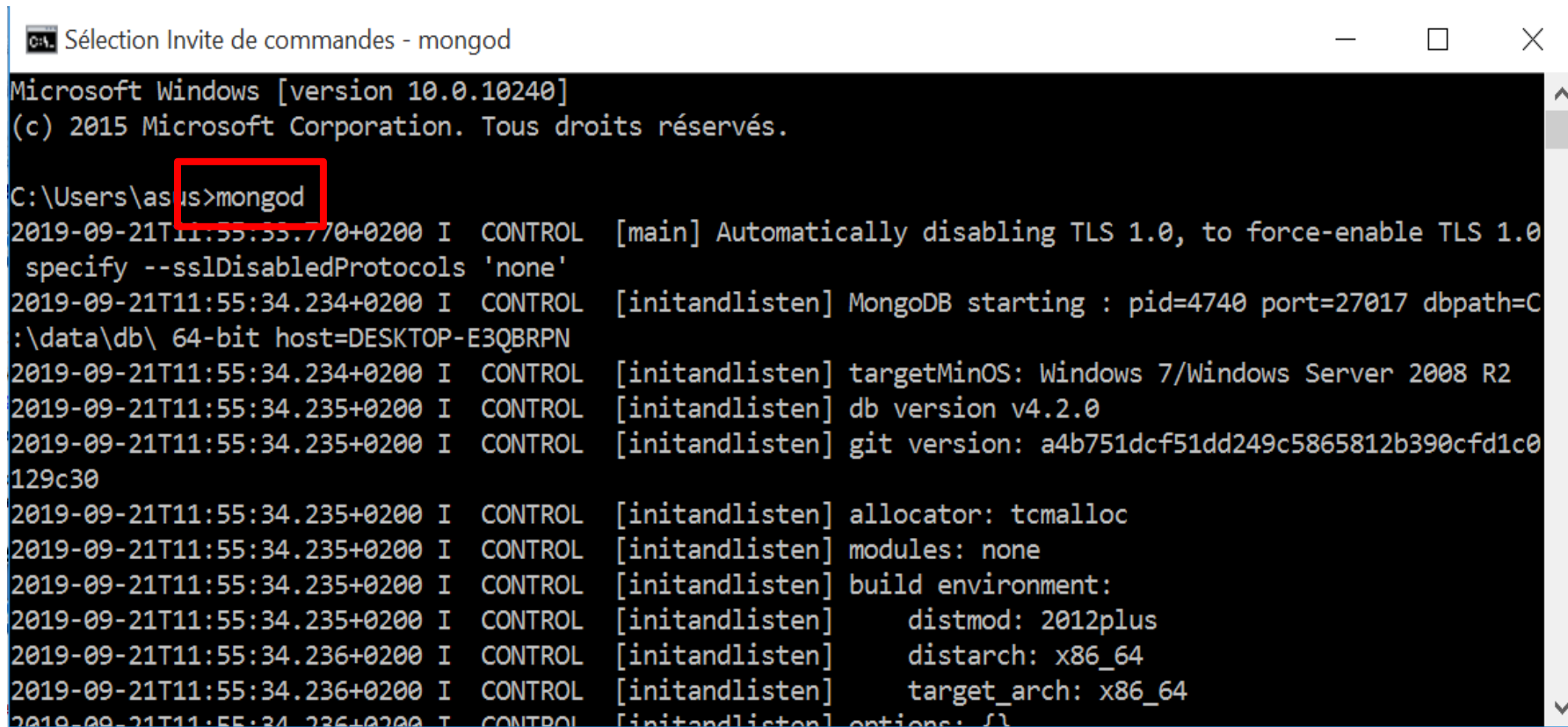
### ➤ Etape 4: Lancement de MongoDB

- Pour lancer MongoDB, exécutez le fichier **mongod.exe**
  - Ceci **lance le processus** de la base de données MongoDB.
- **mongod** est le **processus primaire** pour le système MongoDB. Il traite les requêtes sur les données, gère l'accès aux données, et effectue des opérations de traitement en arrière-plan.



## Installation & Connexion à MongoDB

### ➤ Etape 4: Lancement de MongoDB



```
Sélection Invite de commandes - mongod
Microsoft Windows [version 10.0.10240]
(c) 2015 Microsoft Corporation. Tous droits réservés.

C:\Users\asus>mongod
2019-09-21T11:55:33.770+0200 I  CONTROL  [main] Automatically disabling TLS 1.0, to force-enable TLS 1.0
specify --sslDisabledProtocols 'none'
2019-09-21T11:55:34.234+0200 I  CONTROL  [initandlisten] MongoDB starting : pid=4740 port=27017 dbpath=C:\data\db\ 64-bit host=DESKTOP-E3QBRPN
2019-09-21T11:55:34.234+0200 I  CONTROL  [initandlisten] targetMinOS: Windows 7/Windows Server 2008 R2
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten] db version v4.2.0
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten] git version: a4b751dcf51dd249c5865812b390cfd1c0129c30
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten] allocator: tcmalloc
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten] modules: none
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten] build environment:
2019-09-21T11:55:34.235+0200 I  CONTROL  [initandlisten]   distmod: 2012plus
2019-09-21T11:55:34.236+0200 I  CONTROL  [initandlisten]   distarch: x86_64
2019-09-21T11:55:34.236+0200 I  CONTROL  [initandlisten]   target_arch: x86_64
2019-09-21T11:55:34.236+0200 I  CONTROL  [initandlisten] options: {}
```

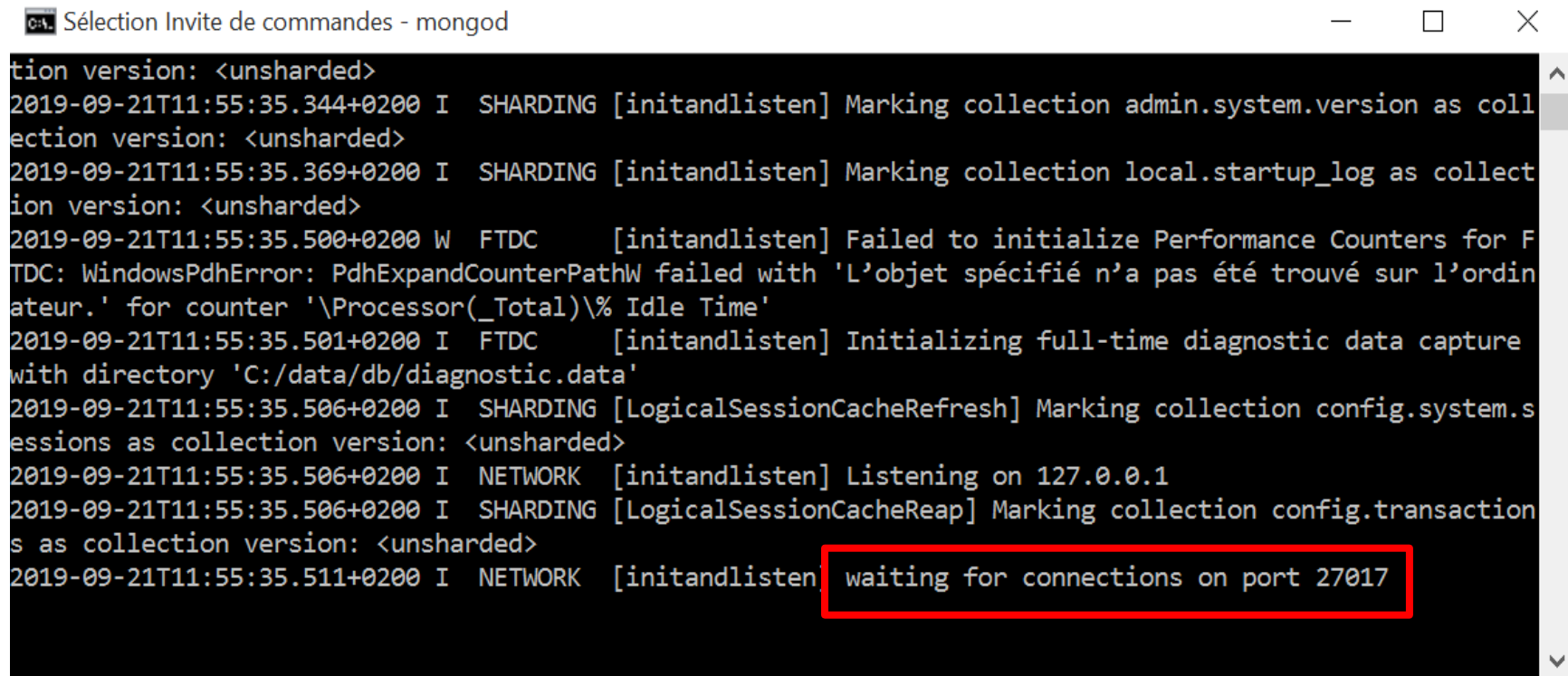


mongoDB

## Installation & Connexion à MongoDB

### ➤ Etape 4: Lancement de MongoDB

Le message «  
**waiting for connections** » sur  
l'invite de  
commandes indique  
que le processus  
mongod.exe a bien  
été exécuté.



```
Sélection Invite de commandes - mongod
tion version: <unsharded>
2019-09-21T11:55:35.344+0200 I SHARDING [initandlisten] Marking collection admin.system.version as coll
ection version: <unsharded>
2019-09-21T11:55:35.369+0200 I SHARDING [initandlisten] Marking collection local.startup_log as collect
ion version: <unsharded>
2019-09-21T11:55:35.500+0200 W FTDC [initandlisten] Failed to initialize Performance Counters for F
TDC: WindowsPdhError: PdhExpandCounterPathW failed with 'L'objet spécifié n'a pas été trouvé sur l'ordin
ateur.' for counter '\\Processor(_Total)\\% Idle Time'
2019-09-21T11:55:35.501+0200 I FTDC [initandlisten] Initializing full-time diagnostic data capture
with directory 'C:/data/db/diagnostic.data'
2019-09-21T11:55:35.506+0200 I SHARDING [LogicalSessionCacheRefresh] Marking collection config.system.s
essions as collection version: <unsharded>
2019-09-21T11:55:35.506+0200 I NETWORK [initandlisten] Listening on 127.0.0.1
2019-09-21T11:55:35.506+0200 I SHARDING [LogicalSessionCacheReap] Marking collection config.transaction
s as collection version: <unsharded>
2019-09-21T11:55:35.511+0200 I NETWORK [initandlisten] waiting for connections on port 27017
```

# Installation & Connexion à MongoDB

### ➤ Etape 5: Connexion à MongoDB

- Pour se connecter à MongoDB, ouvrez une autre fenêtre d'invite de commandes et tapez la commande **mongo**.
- La connexion ne peut réussir que si le service MongoDB est lancé.
- Par défaut, **mongo** recherche un serveur de base de données sur le port **27017** sur l'interface locale.





# Installation de MongoDB

**MongoDB est  
prêt pour être  
utiliser**

```
Invite de commandes - mongo
Microsoft Windows [version 10.0.10240]
(c) 2015 Microsoft Corporation. Tous droits réservés.

C:\Users\asus>mongo
MongoDB shell version: 4.2.0
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("d38ff455-714b-4ff2-9e12-6cdd19f12243") }
MongoDB server version: 4.2.0
Server has startup warnings:
2019-09-21T12:19:41.208+0200 I CONTROL [initandlisten]
2019-09-21T12:19:41.208+0200 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.
2019-09-21T12:19:41.210+0200 I CONTROL [initandlisten] **      Read and write access to data and configuration is
unrestricted.
2019-09-21T12:19:41.214+0200 I CONTROL [initandlisten]
2019-09-21T12:19:41.216+0200 I CONTROL [initandlisten] ** WARNING: This server is bound to localhost.
2019-09-21T12:19:41.219+0200 I CONTROL [initandlisten] **      Remote systems will be unable to connect to this se
rver.
2019-09-21T12:19:41.222+0200 I CONTROL [initandlisten] **      Start the server with --bind_ip <address> to specif
y which IP
2019-09-21T12:19:41.226+0200 I CONTROL [initandlisten] **      addresses it should serve responses from, or with -
bind_ip_all to
2019-09-21T12:19:41.229+0200 I CONTROL [initandlisten] **      bind to all interfaces. If this behavior is desired
, start the
2019-09-21T12:19:41.232+0200 I CONTROL [initandlisten] **      server with --bind_ip 127.0.0.1 to disable this war
ning.
2019-09-21T12:19:41.236+0200 I CONTROL [initandlisten]
---
Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make product
improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
>
```

# Interface Graphique de MongoDB



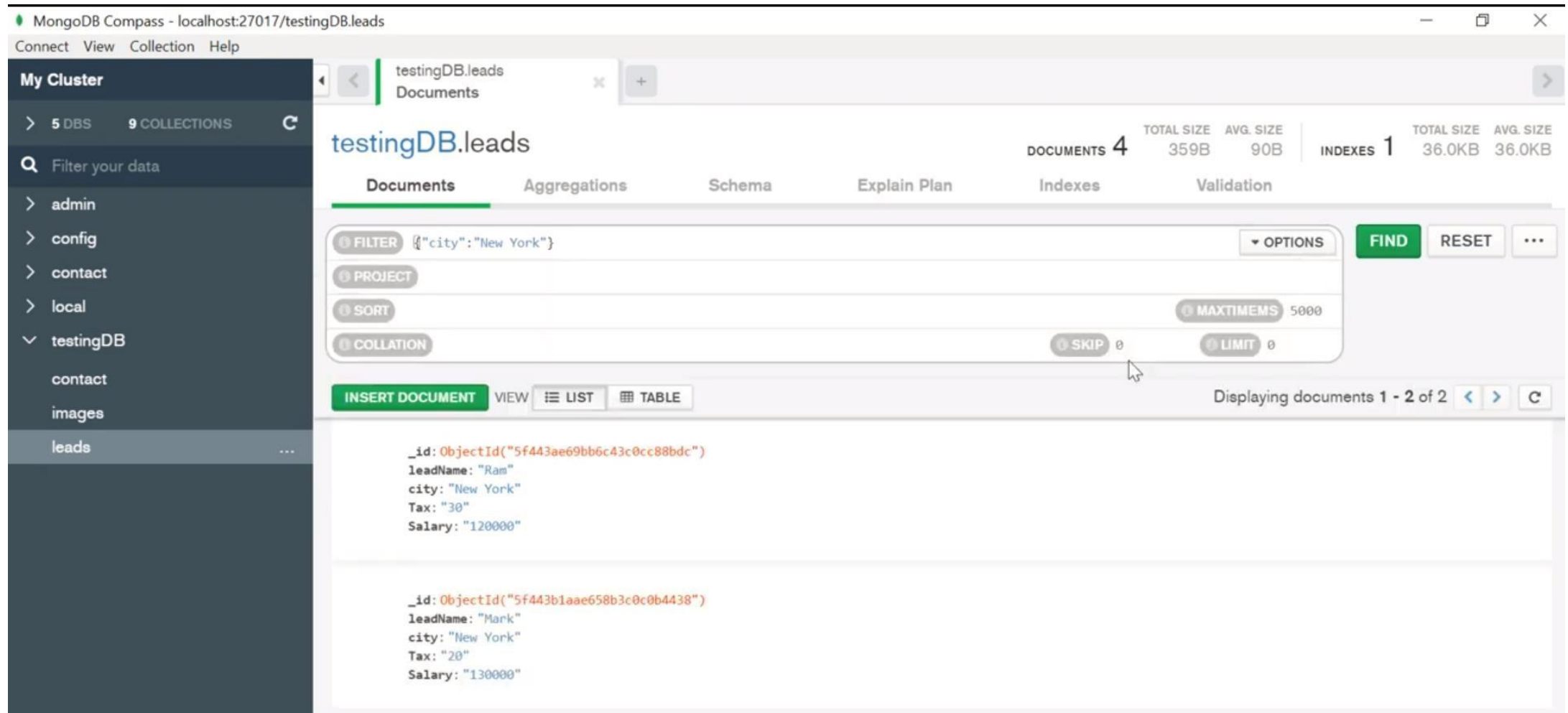
mongoDB



## MongoDB GUI : Examples

- MongoDB Compass
- Robo 3T
- Studio 3T
- Mongo Booster
- ...

## MongoDB GUI : MongoDB Compass

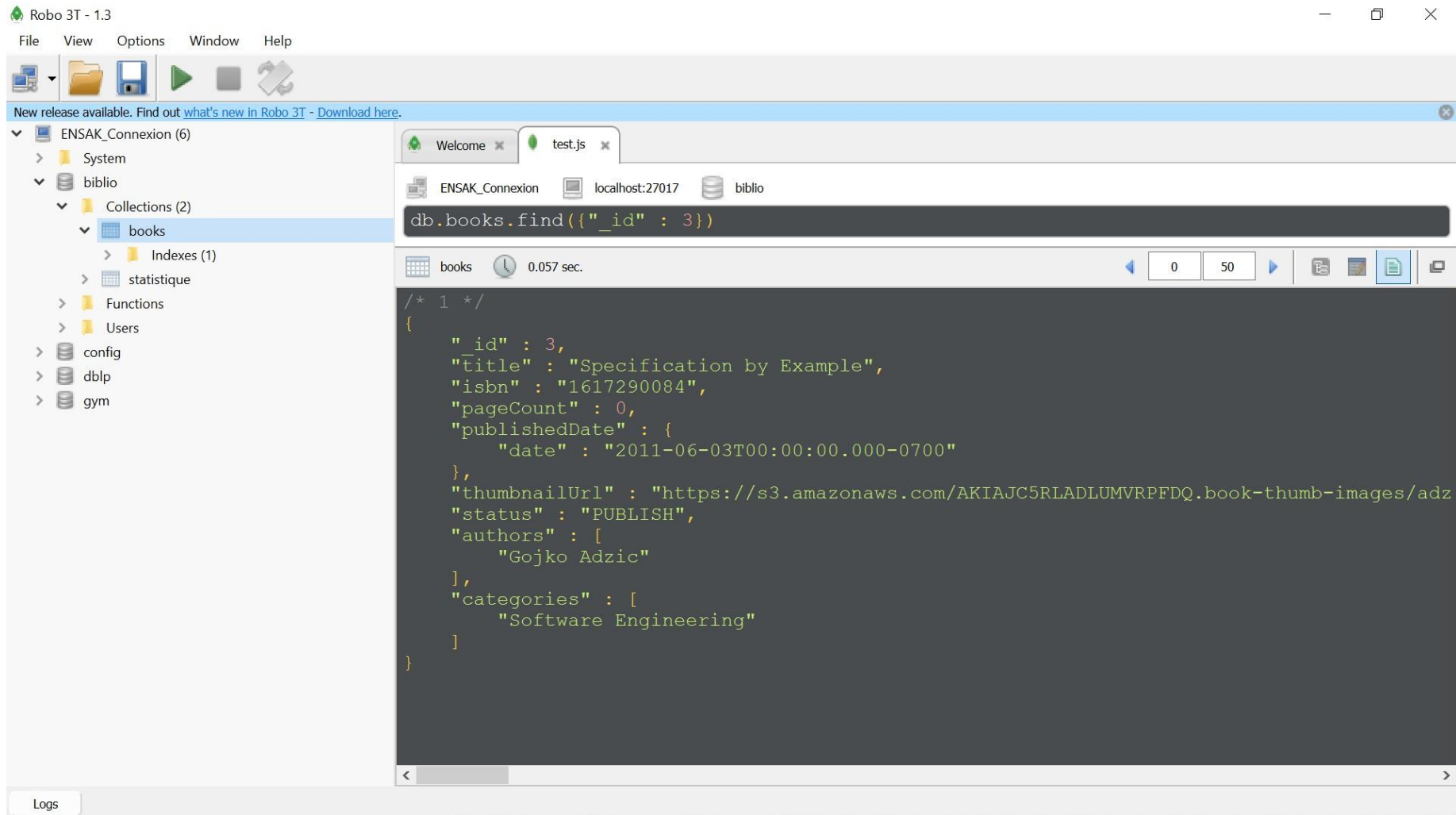


## Robo 3T : Généralité

- **Robo 3T** (anciennement *Robomongo*) est une interface utilisateur de bureau très populaire pour vos déploiements d'hébergement MongoDB.
- Permet d'interagir avec les données par le biais d'**indicateurs visuels** plutôt que d'une interface textuelle.
- Cet outil open source prend en charge plusieurs plates-formes et **intègre en réalité le shell mongo** dans son interface afin de fournir une **interaction à la fois avec le shell et l'interface graphique**.



## Robo 3T : Généralité



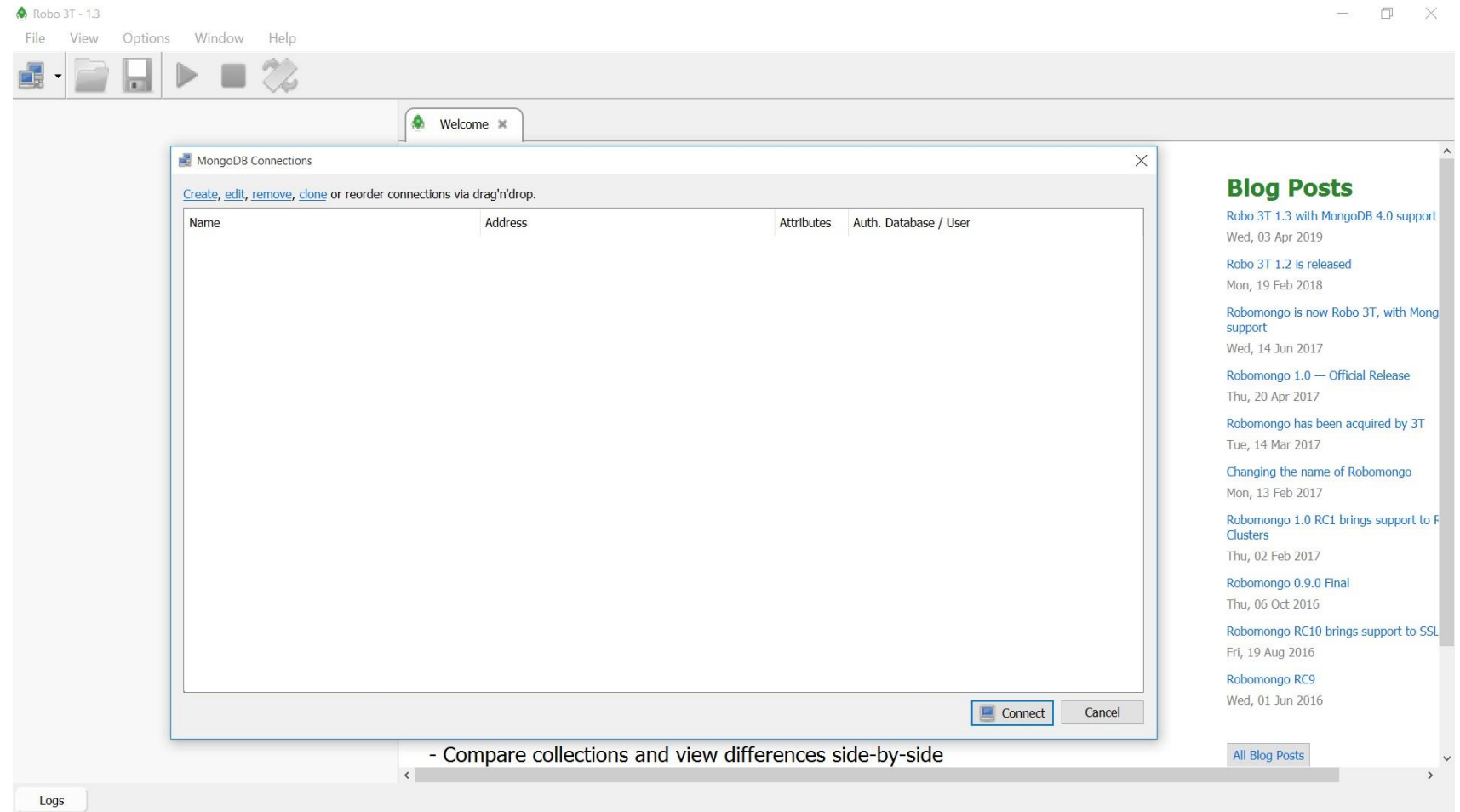
mongoDB

# Interface Graphique de MongoDB

Robo 3T est téléchargeable via le lien suivant: <https://robomongo.org/download>

## Remarque:

Tout d'abord, On doit **démarrer le serveur** avec la commande mongod, et par la suite on **lance Robo 3T**.



## Robo 3T : Création d'une nouvelle connexion

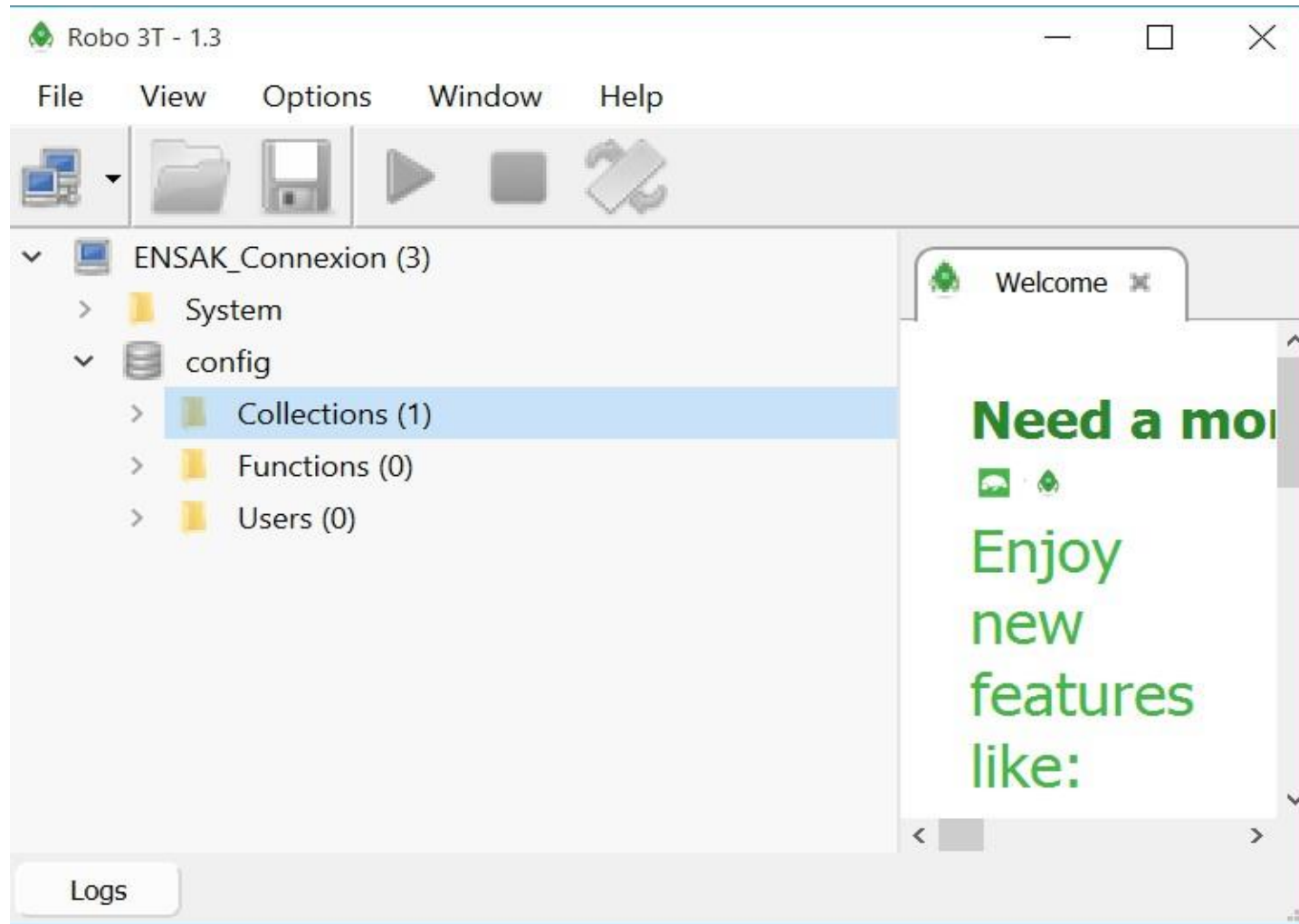
(1) **Create**, edit, remove, clone or reorder connections via drag'n'drop.

| Name                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Address | Attributes | Auth. Database / User |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------------|-----------------------|
| <div><div>Connection Settings</div><div><div>Connection</div><div>Authentication</div><div>SSH</div><div>SSL</div><div>Advanced</div></div><div>Type: Direct Connection</div><div>Name: ENSAK_Connexion (2)</div><div>Address: localhost : 27017</div><div>Specify host and port of MongoDB server. Host can be either IPv4, IPv6 or domain name.</div><div>From SRV Import connection details from MongoDB SRV connection string</div><div>Test (3) Save Cancel</div></div> |         |            |                       |

(4) **Connect** Cancel



## Robo 3T : Création d'une nouvelle connexion



# Opérations de Base sur le Shell



mongoDB



## Commandes de Base

- **mongod** : démarrer le serveur
- **mongo** : démarrer une connexion client (démarrer le shell javascript)
- **mongo nom\_de\_la\_BD** : se connecter avec le nom de la base de données voulue.
- **db** : Pour avoir le nom de la base de données courante.
- **show dbs** : Pour avoir la liste des bases de données.



# Création & Suppression d'une BD

- **Création :**

`use nom_de_la_BD` : si la base de données existe, on y accède directement. Dans le cas contraire, elle sera créée avant qu'on y accède.

- **Suppression :**

`db.dropDatabase()`



# Opérations sur les Collections

### – Création d'une collection :

#### ▪ Méthode 1: Directement

**db.createCollection('nom\_collection')**

#### ▪ Méthode 2: En insérant un document

**db.nom\_collection.insert({clé1:"val1", clé2: "val2" , ...})**

### – Lister les collections existantes : **show collections**

### – Suppression d'une collection : **db.nom\_Collection.drop()**

*(Retourne true ou false selon le résultat)*



## Opérations sur les Documents

### 1. L'ajout d'un document

- **Méthode 1:**

```
db.nom_collection.insert({clé1:"val1", clé2: "val2" , ...})
```

- **Méthode 2:**

```
var nom_variable = ({clé1 : 'val1', clé2: 'val2' , ...})
```

```
db.nom_Collection.save(nom_variable)
```

*save* : ajoute l'élément s'il n'existe pas, sinon elle le modifie en écrasant le contenu précédent.



# Opérations sur les Documents

## 1. L'ajout d'un document (suite)

- Pour vérifier que l'ajout à eu lieu

`db.nom_Collection.find()`

- Pour faire plusieurs insertions en même temps

```
db.nom_Collection.insert( [  
    {clé1:'val1', clé2:'val2', ...},  
    ...,  
    {cléN:'valN', cléM:'valM', ...}  
])
```



## Opérations sur les Documents

### 2. La suppression d'un document

- **Méthode 1: avec *remove***

La méthode `remove()` s'utilise en précisant des paramètres de la manière suivante :

```
db.nom_collection.remove({clé:"valeur"})
```



# Opérations sur les Documents

## 2. La suppression d'un document

- **Méthode 2: avec *delete***

- `db.nom_collection.deleteMany({})`: Supprime tous les documents de la collection.
- `db.nom_collection.deleteMany({<field1>: <value1>, ... })`: supprime les documents ayant les spécifications précisées dans le filtre.

**Remarque: `deleteOne()`** Supprime un seul document



## Opérations sur les Documents

### 3. La sélection d'un document

- Récupérer tous les documents d'une collection

`db.nom_Collection.find()`

- Rechercher selon des critères

`db.nom_Collection.find({clé: "valeur", ... })`

### 4. Compter le nombre de documents

`db.nom_Collection.find({clé: "valeur" ... }).count()`



mongoDB



# Opérations sur les Documents

## 5. Opérations sur l'affichage de documents

- **Trier le résultat de recherche dans l'ordre croissant**  
`db.nom_Collection.find().sort({clé: 1})`
- **Trier le résultat de recherche dans l'ordre décroissant**  
`db.nom_Collection.find().sort({clé: -1})`
- **Sauter quelques documents (ne pas les afficher)**  
`db.nom_Collection.find().skip(nbr_de_doc_à_masqués)`
- **Limiter le nombre de documents à afficher**  
`db.nom_Collection.find().limit(nbr_de_doc_à_affichés)`



# Opérations sur les Documents

## 5. Opérations sur l'affichage de documents *(suite)*

- **Afficher seulement le premier document**  
`db.nom_Collection.findOne()`
- **Afficher le résultat au format JSON**  
`db.nom_Collection.find().pretty()`



# Opérations sur les Documents

## 5. Opérations sur l'affichage de documents *(suite)*

### Remarque :

La fonction *find* opère sur 2 paramètres :

- Paramètre 1 : Filtre de sélection
- Paramètre 2 : Lister les attributs à affichés dans le résultat.



# Opérations sur les Documents

## 5. Opérations sur l'affichage de documents *(suite)*

- **Afficher seulement quelques champs**

```
db.personne.find({}, {nom:1})
```

*(ça affiche le nom de tous les personnes ainsi que leurs identifiants)*

- **Et si on ne veut pas afficher les ids**

```
db.personne.find({}, {nom:1, _id:0})
```



mongoDB

# Opérations sur les Documents

## 6. Mettre à jour un document

Quelle commande pour faire la modification ?

`db.nom_Collection.update()`

Elle prend au moins deux paramètres :

- **1<sup>er</sup> paramètre** : document(s) concernés par la modification,
- **2<sup>ème</sup> paramètre** : le type de modification



mongoDB

# Opérations sur les Documents

## 6. Mettre à jour un document (suite)

Qu'est ce qu'on peut faire comme modification?

- **\$set** : pour modifier la valeur d'un champ. Si le champ existe, il est mis à jour. Sinon, il est ajouté au document.
- **\$unset** : pour supprimer un champ.
- **\$inc** : pour incrémenter la valeur d'un champ.
- **\$mul** : pour multiplier l'ancienne valeur d'un champ par la valeur spécifiée.
- **\$rename** : pour renommer un champ.
- ...



# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

- **Modification d'un document**

```
db.nom_Collection.update({clé1 : 'val1', ... },  
                           { $set: {cléN : 'valN' } })
```

Il existe d'autres méthodes de mise-à-jour:

- `updateOne()` : pour modifier le premier enregistrement de la sélection (par défaut)
- `updateMany()` : pour modifier toute la sélection



# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

### ▪ Modification d'un document (suite)

#### Exemple

```
db.personne.update( {prenom: "Jean"}, { $set: {ville: 'Nantes'} })
```

- On commence par sélectionner les documents dont le champ prenom contient comme valeur Jean.
- Pour ce document, on mettra la valeur Nantes pour le champ ville s'il existe. Sinon, le champ ville sera créé avec la valeur Nantes.





# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

### ■ Modification d'un document (suite)

- Dans l'exemple précédent, seul le premier document de la collection `personne` sera concerné par la modification.
- Pour que la fonction `update()` modifie plusieurs documents, il faut ajouter l'option '`multi`':

### Exemple:

```
db.personne.update( {prenom:"Jean"}, {$set:{ville:'Nantes'},  
                    {multi:true} })
```



# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

### ■ Supprimer un champ d'un document

```
db.personne.update( {prenom: "Jean"}, {$unset: {prenom:1} })
```

- On commence par sélectionner les documents dont le champ prenom contient comme valeur Jean.
- Pour ce(s) document(s), on supprimera le champ prénom s'il existe.



# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

### ▪ Incrémenter un champ d'un document

```
db.personne.update( {prenom: "Jean"}, {$inc: {age:10}} )
```

- On commence par sélectionner les documents dont le champ prenom contient comme valeur Jean.
- Pour ce(s) document(s), on incrémentera de 10 l'age s'il existe. Sinon, le champ age sera créé avec la valeur 10.



# Opérations sur les Documents

## 5. Mettre à jour un document (suite)

- Renommer un champ d'un document

```
db.personne.updateMany({},{$rename:{nom: "name" } })
```

- On commence par sélectionner tous les documents.
- Pour ces documents, le champ nom sera renommé name.



mongoDB

# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau

### ▪ Liste des opérations:

- **\$push** : permet d'ajouter des éléments à la fin d'un tableau si ce dernier existe et dans le cas contraire, il le crée.
- **\$each**: permet d'ajouter des informations de façon consécutive pour un champ donné. Mais il s'utilise avec l'opérateur *\$push*.
- **\$pop** : pour supprimer le premier ou le dernier élément d'un tableau
- **\$pull** : pour supprimer une ou plusieurs valeurs d'un tableau



# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau(suite)

### ▪ Liste des opérations: (suite)

- **\$pullAll** : pour supprimer tous les éléments d'un tableau
- **\$position** : à utiliser avec push pour indiquer la position d'insertion dans un tableau.
- **\$slice** : à utiliser avec *\$push* pour préciser les éléments à garder dans un tableau (*si le nbr est négatif ie on garde juste les derniers éléments du tableau et inversement*)
- **\$sort** : à utiliser avec push pour ordonner les éléments d'un tableau.

# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau (suite)

Considérons le document suivant :

```
db.personne.insert({  
  _id : 5,  
  prenom : 'Jean',  
  sport: [ 'foot', 'hand', 'tennis'] })
```



## Opérations sur les Documents

### 6. Opérations sur un champ tableau(suite)

- Ajouter un nouveau sport au tableau

```
db.personne.update({_id: 5}, { $push: {"sport": "basket"}})
```

- Ajouter plusieurs sports avec une seule requête

```
db.personne.update ({_id: 5}, { $push: {  
    "sport": { $each: ['basket','sky'] } } })
```

*\$push* : ajoute naturellement l'élément après le dernier  
Élément du tableau





# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau (suite)

- Ajouter un élément à une position précise dans le tableau

```
db.personne.update ( {_id: 5}, { $push: {  
                                "sport": { $each: ['volley'], $position: 1 } } })
```

- Ceci rajoute l'élément volley à la position 2 du tableau sport (*la première position est d'indice 0*)
- Les autres éléments seront décalés



# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau (suite)

- Supprimer le premier élément d'un tableau

```
db.personne.update( {_id: 5}, {$pop: { sport: -1 }})
```

- Supprimer le dernier élément d'un tableau

```
db.personne.update( {_id: 5}, {$pop: { sport: 1 }})
```

- Supprimer un élément quelconque d'un tableau

```
db.personne.update( {_id: 5}, {$pull: {"sport": "foot"}})
```

=> supprime l'élément foot du tableau sport.



# Opérations sur les Documents

## 6. Opérations sur un champ de type tableau (suite)

- Préciser les éléments à garder dans un tableau (suite)

```
db.personne.update( { _id: 5 }, {  
  $push: { sport: {  
    $each: ['sky', 'volley', 'hockey'],  
    $slice: -5  }  
  }  
})
```

=> Le résultat est : hand, tennis, sky, volley, hockey



# Opérations sur les Documents

## 7. Opérations sur un champ de type '*tableau de document*'

Considérons le document suivant :

```
db.personne.insert({  
  _id : 10,  
  prenom : 'Sara',  
  notes : [ {'BD NoSQL': 17, 'coefficient': 4},  
            {'Android': 12, 'coefficient': 2} ]  
})
```



mongoDB

# Opérations sur les Documents

## 7. Opérations sur un champ de type '*tableau de document*' (suite)

- Ajouter un nouveau document au tableau

```
db.personne.update( { _id: 10 },  
  { $push:  
    { notes : { 'compilation': 15, 'coefficient': 1 } }  
  }  
)
```



# Opérations sur les Documents

## 7. Opérations sur un champ de type '*tableau de document*' (suite)

- Supprimer un document du tableau

```
db.personne.update( { _id: 10 },  
  { $pull:  
    { notes : { 'compilation': 15, 'coefficient': 1 } }  
  }  
)
```



## Opérations sur les Documents

### 7. Opérations sur un champ de type '*tableau de document*' (suite)

- Chercher un document selon une valeur dans son tableau d'objet

```
db.personne.find(  
    { "notes.Android" : {$ne:17} }  
)
```

↑  
\$ne = not equal = différent de



# Opérateurs de Logique/Comparaison

Comme pour les bases de données relationnelles, la base de données NoSQL MongoDB est constitué aussi d'opérateurs de:

- **Comparaison**
- **Logiques**





# Opérateurs de Logique/Comparaison

## 1. Les opérateurs de comparaison

- **\$gt** : greater than (supérieur à)
- **\$gte** : greater than or equal (supérieur ou égal)
- **\$lt** : less than (inférieur à)
- **\$lte** : less than or equal (inférieur ou égal)
- **\$eq** : equal (égal à)
- **\$ne** : not equal (différent de )
- **\$in** : dans (un tableau...)
- **\$nin** : not in (pas dans)



## Opérateurs de Logique/Comparaison

### 2. Les opérateurs logiques

- **\$and** : ET
- **\$or** : OU
- **\$not** : le NON logique
- **\$nor** : OU exclusif
- **\$exists** : tester l'existence d'un champs. Elle peut prendre deux valeurs: true ou false.



# Opérateurs de Logique/Comparaison

## 1. Les opérateurs de comparaison

**Exemple 1:** sélectionner les personnes âgés de plus de 20 ans

```
db.personne.find( {"age": { $gt:20 } })
```

**Exemple 2:** sélectionner les personnes dont l'âge est entre 20 et 30 ans

```
db.personne.find(  
  { $and:  
    [ { age: { $gte:20 } }, { age: { $lte:30 } } ]  
  }  
)
```



## Opérateurs de Logique/Comparaison

### 2. Les opérateurs logiques

**Exemple 1:** Afficher les personnes dont le champ name existe

```
db.personne.find( { name: { $exists: true } } )
```

**Exemple 2:** Afficher les personnes dont l'age est divisible par 5

```
db.personne.find( { age: { $mod: [ 5, 0 ] } } )
```



mongoDB

# Opérations Avancées sur le Shell



mongoDB

# Expressions régulières

On peut utiliser les expressions régulières dans une requête MQL.

- **chercher les personnes dont le nom commence par w :**

```
db.personne.find( {name: /^w/ } )
```

⇒ **Explication:**

- les deux / pour indiquer le début et la fin de l'expression régulière.
- ^ pour indiquer par quoi commence le mot cherché.



### Expressions régulières

- chercher les personnes dont le nom se termine par k :

```
db.personne.find( {name:/k$/} )
```

- chercher les personnes dont le nom commence par a ou par n :

```
db.personne.find( {name:/^[an]/} )
```

- chercher les personnes dont le nom commence par une lettre comprise entre a et n:

```
db.personne.find( {name:/^[a-n]/} )
```



## Expressions régulières: Autres symboles utilisés

- **x?** : pour indiquer que la lettre x est facultative. Elle peut y être 0 ou 1 fois.
- **x+** : pour indiquer que la lettre x est obligatoire. Elle peut y être 1 ou plusieurs fois.
- **x\*** : pour indiquer que la lettre x est facultative. Elle peut y être 0, 1 ou plusieurs fois.
- **x{2,4}** : pour indiquer que la lettre x doit se répéter au moins deux fois et au plus 4 fois.
- **.** : un caractère quelconque
- **|** : le ou logique





## Expressions régulières: Autres symboles utilisés

|                            |                                          |
|----------------------------|------------------------------------------|
| <code>[ ^x ]</code>        | Tous caractères sauf $x$                 |
| <code>[ aeiou ]</code>     | Tous caractères aeiou                    |
| <code>[ a-z , A-Z ]</code> | Tous caractères majuscules et minuscules |
| <code>[ ^aeiou ]</code>    | Tous caractères sauf aeiou               |

|                    |                                     |
|--------------------|-------------------------------------|
| <code>*</code>     | Répète un nombre de fois quelconque |
| <code>+</code>     | Répète au moins une fois            |
| <code>?</code>     | Répète zéro ou une fois             |
| <code>{n}</code>   | Répète $n$ fois                     |
| <code>{n,m}</code> | Répète entre $n$ et $m$ fois        |
| <code>{n,}</code>  | Répète au minimum $n$ fois          |



## Les agrégations

- L'opérations d' **agrégations** traitent les enregistrements de données et renvoient les résultats calculés.
- Les opérations d'agrégation regroupent les valeurs de plusieurs documents et peuvent effectuer diverses opérations sur les données groupées pour renvoyer un seul résultat.
- Exemple : Group By, SUM, MIN, MAX, AVERAGE...



## Les agrégations

**db.nom\_collection.aggregate** (pipeline)

| Paramètre       | Détails                                                               |
|-----------------|-----------------------------------------------------------------------|
| <b>pipeline</b> | array (Une séquence d'opérations ou d'étapes d'agrégation de données) |



### Les agrégations : Exemples des pipelines

➤ **\$match**: permet de filtrer les documents selon la condition spécifiée

➤ **\$project**: pour sélectionner les champs que l'on souhaite conserver dans le résultat

➤ **\$group** : Permet de regrouper des documents comme un *GROUP BY* en SQL

➤ **\$out** : pour enregistrer le résultat dans une nouvelle collection

➤ **\$sample**: Permet de sélectionner aléatoirement un nombre de documents spécifiée dans la requête

➤ **\$unwind** : Permet de décomposer un tableau en autant de documents que d'élément.

## Les agrégations : Exemples

Considérons la collection suivante (étudiants) :

```
{"_id" : 1, "nom" : "wick", "notes": [10, 15, 12], "age" : 19 }
```

```
{"_id" : 2, "nom" : "bob", "notes": [18, 8, 12], "age" : 35 }
```

```
{"_id" : 3, "nom" : "wolf", "notes": [7, 6, 13], "age" : 25 }
```

```
{"_id" : 4, "nom" : "green", "notes": [18, 16, 9], "age" : 22 }
```



## Les agrégations : Exemples

**Exemple 1:** `db.etudiants.aggregate( [`  
    `{ $project :`  
        `{ _id: "$nom",`  
          `moyenne: {$avg: "$notes"}`  
        `}`  
    `}]`);

Le résultat

```
{ "_id" : "wick", "moyenne" : 12.333333333333334 }  
{ "_id" : "bob", "moyenne" : 12.666666666666666 }  
{ "_id" : "wolf", "moyenne" : 8.666666666666666 }  
{ "_id" : "green", "moyenne" : 14.333333333333334 }
```

## Les agrégations : Exemples

**Exemple 2:** `db.etudiants.aggregate(  
 [ {$project: { _id: "$nom",  
 moyenne: {$avg: "$notes"}  
 }  
 }, { $out : "moyennes" }  
 ]  
);`

- **Vérifier la création de la collection moyennes:**  
show collections; et db.moyennes.find();

### Les agrégations : Exemples

**Exemple 3**: `db.etudiants.aggregate( [`  
                                  `{ $sample: { size: 3 } } ]`  
                                  `)`

**Le résultat :**

Il choisit aléatoirement trois documents de la collection



## Les agrégations : Exemples

**Exemple 4:** `db.etudiants.aggregate([ { $unwind : "$notes" } ])`

### Le résultat

```
{ "_id" : 1, "nom" : "wick", "notes" : 10, "age" : 19 }
{ "_id" : 1, "nom" : "wick", "notes" : 15, "age" : 19 }
{ "_id" : 1, "nom" : "wick", "notes" : 12, "age" : 19 }
{ "_id" : 2, "nom" : "bob", "notes" : 18, "age" : 25 }
{ "_id" : 2, "nom" : "bob", "notes" : 8, "age" : 25 }
{ "_id" : 2, "nom" : "bob", "notes" : 12, "age" : 25 }
{ "_id" : 3, "nom" : "wolf", "notes" : 7, "age" : 35 }
{ "_id" : 3, "nom" : "wolf", "notes" : 6, "age" : 35 }
{ "_id" : 3, "nom" : "wolf", "notes" : 13, "age" : 35 }
{ "_id" : 4, "nom" : "green", "notes" : 18, "age" : 22 }
{ "_id" : 4, "nom" : "green", "notes" : 16, "age" : 22 }
```

### Les agrégations : **Exemples**

Considérons la collection books suivante:

```
{ "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 }  
{ "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 }  
{ "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 }  
{ "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 }  
{ "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
```

### Les agrégations : Exemples

```
db.books.aggregate( [  
    { $match : { author : "Dante" } }  
]);
```

Le résultat :

```
{ "_id" : 8751, "title" : "The Banquet", "author" :  
  "Dante", "copies" : 2 }  
{ "_id" : 8752, "title" : "Divine Comedy", "author"  
  : "Dante", "copies" : 1 }  
{ "_id" : 8645, "title" : "Eclogues", "author" : "  
  Dante", "copies" : 2 }
```

**Documentation Officielle:**

**<https://docs.mongodb.com/manual>**



mongoDB